MIT Academy of Engineering, Alandi(D), Pune
Department of CSE AIML

Assignment - 5

Implement Stack using two Queues

Student Name: Vivek Vaidya

PRN: 202501110081

Batch: B1

Problem Statement:

Implementing a Stack Using Two Queues

1. Conceptual Understanding: Stack vs. Queue

A Stack is a Last-In-First-Out (LIFO) data structure where elements are added and removed from the same end, called the "top". A Queue is a First-In-First-Out (FIFO) data structure where elements are added at the back (enqueue) and removed from the front (dequeue). Both are linear data structures. To implement a Stack using Queues, we must reverse the order of incoming elements so that the most recently added element is always accessible at the front of the primary queue.

2. Implementation of Operations (C++)

```
#include <queue>

#include <iostream>

#include <stdexcept>


class StackUsingTwoQueues {

private:

  std::queue<int> q1;

  std::queue<int> q2;
```

```
public:
    void push(int x) {
        q2.push(x);
        while (!q1.empty()) {
            q2.push(q1.front());
            q1.pop();
        }
        std::queue<int> temp = q1;
        q1 = q2;
        q2 = temp;
    }

    int pop() {
        if (q1.empty()) {
            throw std::out_of_range("pop from empty stack");
        }
        int val = q1.front();
        q1.pop();
        return val;
    }

    int top() {
        if (q1.empty()) {
            throw std::out_of_range("top from empty stack");
        }
        return q1.front();
```

```
    }

    bool empty() {
        return q1.empty();
    }
};

int main() {
    StackUsingTwoQueues stack;
    stack.push(10);
    stack.push(20);
    stack.push(30);

    std::cout << "Top element: " << stack.top() << std::endl;
    std::cout << "Popped element: " << stack.pop() << std::endl;
    std::cout << "Top element after pop: " << stack.top() << std::endl;
    std::cout << "Is stack empty? " << (stack.empty() ? "true" : "false") << std::endl;

    return 0;
}
```

3. Practical Application & Problem Solving

Implementing stacks using queues is a classic algorithmic problem frequently used in systems design and interviews to test a developer's mastery of abstract data types (ADTs).

Real-World Relevance: In software systems where a component only provides queue APIs (such as message brokers or asynchronous task buffers like RabbitMQ or AWS SQS), you may occasionally need LIFO semantics—for instance, processing the most recent user action first (like an Undo/Redo mechanism or browser history navigation). This conversion pattern proves how underlying data structures can be adapted to fit strict architectural requirements.

4. Analysis of Time and Space Complexity

For the push-costly approach implemented above:

Operation	Time Complexity	Justification
push(x)	O(N)	Every time a new element is pushed, it is added to q2, and all N existing elements from q1 are transferred over to q2.
pop()	O(1)	The top element of the stack is always kept at the front of q1, allowing immediate retrieval and removal.
top()	O(1)	Direct access to the front element of q1.
empty()	O(1)	Simple size check on q1.

Space Complexity: O(N) as we store a total of N elements across both queues.

5. Execution and Output Verification

When compiled and run, the program produces the following accurate output:

